UNIVERSIDAD DE LA SABANA

Departamento de Ingeniería

Maestría en Arquitectura de Software

Observabilidad en Ambientes Productivos

UNA PETICIÓN, TRES SEÑALES

Laboratorio 2.2: pipeline de observabilidad con OpenTelemetry sobre dos microservicios Spring Boot

Caso de estudio: servicio de pedidos e inventario instrumentado con el agente Java de OpenTelemetry, con trazas, métricas y logs correlacionados en Grafana y medición del overhead de instrumentar

Fabián Andrés Rojas García

Jorge Eliécer Rojas Quiñones

Juan Esteban Gómez Roa

Bogotá, Colombia · 2026

1. Introducción y objetivos

El laboratorio pide construir un sistema distribuido que genere métricas, logs y trazas de una misma operación de negocio, y montar el pipeline que las lleve a un lugar donde se puedan consultar juntas. El repositorio del docente (`oTelLabs-main`, en Python con FastAPI) nos sirvió para entender el problema, pero no lo modificamos y no es lo que entregamos: la implementación es propia.

Trabajamos con dos microservicios en Spring Boot 3.5.16 y Java 21. `service-a` busca un pedido en MySQL y le pregunta a `service-b` por el inventario del SKU correspondiente. La operación que usamos de demostración es `GET /api/orders/{orderId}`, que responde si hay stock suficiente (`available >= quantity`). La base se crea con cinco pedidos de prueba, de `ord-001` a `ord-005`.

Todas las cifras del informe salen de la corrida local del 9 de octubre de 2026. Los despliegues en GKE y ECS quedaron preparados, pero no los ejecutamos porque en este entorno no teníamos credenciales de GCP ni de AWS, y tampoco el binario de Terraform.

Lo que quedó funcionando en local:

- Trazas distribuidas que van desde la petición HTTP hasta JDBC, pasando por la llamada entre servicios.
- Métricas HTTP, de la JVM y de negocio en Prometheus, con un dashboard en Grafana.
- Logs en JSON con `trace_id` y `span_id`, consultables en Loki y enlazados con Tempo.
- El OpenTelemetry Collector 0.162.0 como único punto de salida para las tres señales.
- Una prueba de carga que compara la misma lógica de negocio con y sin el agente Java.

Lo que quedó listo, pero sin `terraform apply`: GKE con Cloud Trace, Cloud Logging y Managed Prometheus, y ECS Fargate con X-Ray, CloudWatch Logs y EMF.

2. Arquitectura

La figura 1 resume el recorrido. El cliente llama a `service-a` por el puerto 8080; este consulta MySQL y llama a `service-b` por el 8081, propagando el encabezado `traceparent` de W3C. Los dos servicios envían trazas, métricas y logs al Collector por OTLP gRPC (puerto 4317) y desde ahí cada señal sigue su camino: las trazas a Tempo, las métricas a Prometheus y los logs a Loki. Grafana consulta los tres.

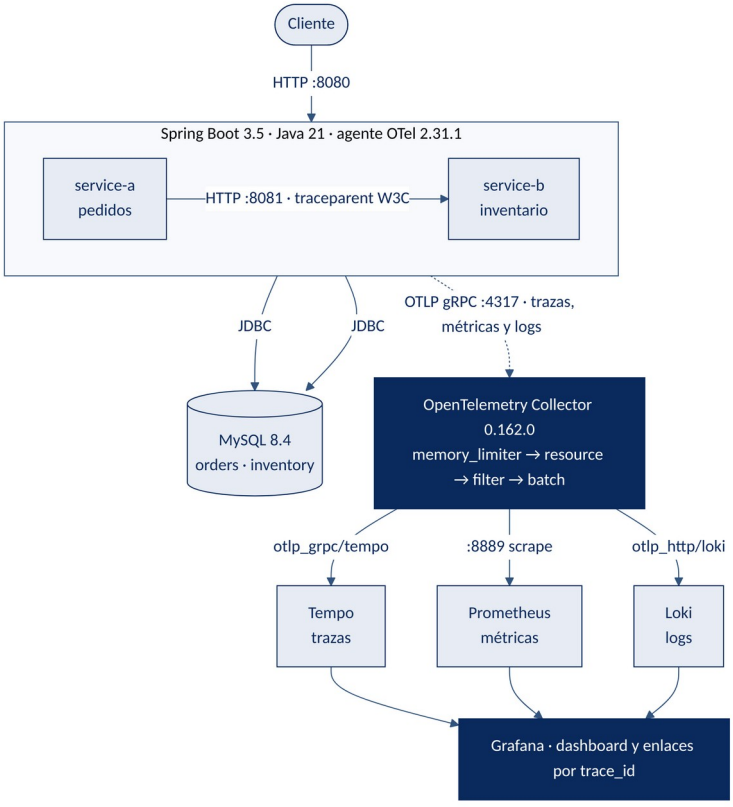


Figura 1. Arquitectura local del laboratorio y recorrido de las tres señales (elaboración propia, 2026).

MySQL 8.4 guarda las tablas `orders` e `inventory`, y accedemos a ellas con `JdbcTemplate`, sin JPA. Como el agente Java ya instrumenta el driver JDBC, no abrimos spans manuales para el SQL, porque solo duplicarían información. Los spans manuales los dejamos para la regla de negocio: `order.compose` en service-a e `inventory.lookup` en service-b.

El agente es `opentelemetry-javaagent` 2.31.1 y la aplicación solo depende de la API 1.65.0. El SDK de runtime lo pone el agente, y así evitamos tener dos SDK exportando al mismo tiempo. Con `OTEL_ENABLED=false` el mismo jar arranca sin `-javaagent`; esa es justamente la diferencia entre el escenario base y el instrumentado en la prueba de carga.

En local el muestreo es `parentbased_traceidratio` con argumento 1.0, o sea que se conservan todas las trazas. Las métricas se exportan cada 15 segundos y los propagadores configurados son `tracecontext` y `baggage`.

Para GCP y AWS no hicimos un segundo diseño de aplicación. Lo que cambia es el exporter del Collector y la plataforma donde corren los contenedores. Las configuraciones están en `observability/collector/collector-gcp.yaml` y `collector-aws.yaml`, y el código de Terraform en `terraform/gcp` y `terraform/aws`.

3. Decisiones de diseño

La tabla 1 recoge las decisiones principales y el motivo de cada una.

Decisión	Motivo
Java y Spring Boot	Es el stack que se pidió para mantenimiento. El ejemplo del docente está en Python.
Agente Java en lugar de SDK embebido	La auto-instrumentación de HTTP y JDBC ya cubre el camino técnico, así que el código de negocio solo abre spans de dominio.
Tempo, sin Jaeger	Grafana ya consulta Tempo. Un segundo almacén de trazas en local no aporta ninguna señal nueva.
Loki para los logs locales	Necesitábamos un backend que Grafana pudiera filtrar por `trace_id`. En GKE ese papel lo cumple Cloud Logging.
Prometheus hace scrape del Collector (:8889)	Las aplicaciones no exponen `/metrics`, con lo que evitamos un segundo registro de Micrometer.
MySQL en el Compose y, en ECS, como sidecar opcional	La traza tiene que mostrar la persistencia. RDS queda detrás de `create_rds=false` para no crear una base de datos de pago sin que alguien lo decida explícitamente.
Sin exporter `googlecloud` en el Compose local	Ese exporter pide credenciales y un proyecto. En el ejemplo del docente estaba activo, apuntando a una ruta de ADC de otra máquina.

Tabla 1. Decisiones de diseño de la implementación (elaboración propia, 2026).

Al revisar el ejemplo del docente con cuidado encontramos dos cosas que había que corregir. La primera es que allí el contador `http_requests_total` se incrementaba dos veces por cada petición exitosa, y con eso un SLI de disponibilidad quedaba cerca del 50 % sin que hubiera errores reales. En nuestra versión el histograma HTTP lo emite el agente, una sola vez por petición, y el dashboard filtra `http_route=~"/api/.*"`. La segunda tiene que ver con el patrón de exclusión del agente, que debe ser `.*/actuator/.*`: un patrón que solo empieza por `/actuator/` no coincide con la URL completa, y el healthcheck terminaba colándose en las series.

También tuvimos en cuenta que los exporters `otlp` y `otlphttp` están deprecados en el Collector 0.162.0, así que la configuración local usa `otlp_grpc/tempo` y `otlp_http/loki`. Los tres archivos YAML pasaron `otelcol-contrib validate`.

4. Los tres pilares

4.1 Trazas

El agente abre el span servidor `GET /api/orders/{orderId}`. Dentro de él, `OrderFulfillmentService` abre `order.compose` como hijo, con los atributos `order.id` y `product.sku`. El SELECT de pedidos aparece como span JDBC y `RestClient` se encarga de propagar `traceparent`. Del lado de service-b se ven el span servidor `GET /api/inventory/{sku}`, el de negocio `inventory.lookup` y el de base de datos `SELECT lab22.inventory`.

Decidimos que un 404 de pedido no marque el span de negocio como error, porque en ese caso lo que pasó es que el cliente pidió un id que no existe. Si el que falla es service-b, `order.compose` sí queda en error. Los dos casos están cubiertos por las pruebas de `mvn test`.

4.2 Métricas

El agente exporta `http.server.request.duration` y las métricas de la JVM, y la aplicación agrega sus propios contadores de negocio: `orders.composed` y los de consultas y reservas de inventario. El Collector los publica en el puerto 8889 con `UnderscoreEscapingWithSuffixes`, de modo que el nombre que Prometheus termina scrapeando para la latencia HTTP es `http_server_request_duration_seconds_count`. Para la CPU del proceso la métrica correcta es `jvm_cpu_recent_utilization_ratio`; el nombre sin `_ratio` no existe en esta traducción y nos estaba dejando el panel vacío.

4.3 Logs

Logback escribe JSON a stdout con `trace_id`, `span_id`, servicio, versión y entorno, y el agente reenvía ese mismo evento por OTLP. En el Collector, el processor `transform/logs` le antepone `trace_id=<32 hex>` al cuerpo del mensaje, y Grafana usa la expresión `trace_id=([0-9a-f]{32})` para convertir ese texto en un enlace.

Un detalle que nos tomó tiempo entender: en Loki el identificador queda guardado como structured metadata y no como etiqueta del stream. La consulta que funciona es `{service_name="service-a"} | trace_id="..."`; tratar `trace_id` como etiqueta de índice no lleva a ningún lado.

5. Collector

El Collector recibe por OTLP en gRPC (4317) y en HTTP (4318). En el pipeline de trazas los processors van en este orden: `memory_limiter` (400 MiB, con picos de 100), `resource`, `filter/health` y `batch` (5 s, 1024 elementos, tope de 2048). El limitador va primero para rechazar carga antes de gastar memoria, y el batch va de último para no empaquetar spans que el filtro igual va a descartar. Los logs pasan además por `transform/logs`.

Para vigilar la salud del propio Collector activamos la extensión `health_check` en el puerto 13133 y las métricas internas en el 8888, que Prometheus scrapea con el job `otel-collector`. El panel de fallos usa `or vector(0)`, porque las series de error no existen hasta que ocurre el primer fallo y, sin ese ajuste, Grafana deja el panel en blanco.

En GCP el Collector exporta trazas y logs con `googlecloud` y métricas con `googlemanagedprometheus`; la cuenta de servicio prevista tiene los roles `cloudtrace.agent`, `logging.logWriter` y `monitoring.metricWriter`. En AWS usa `awsxray`, `awscloudwatchlogs` y `awsemf`. Como la task de Fargate no tiene bind mount, la imagen del Collector ya trae `collector-aws.yaml` desde el build.

6. Evidencia de una petición real

El 9 de octubre de 2026, con `OTEL_ENABLED=true` y después del benchmark, hicimos `GET /api/orders/ord-001`. La respuesta fue:

- pedido `ord-001`, SKU `LAPTOP-X1`, cantidad 1, estado `delivered`, cliente `cust-100`;
- inventario con 15 unidades disponibles y 2 reservadas en el almacén `WH-BOGOTA`;
- `stockSufficient=true`;
- `traceId=77fa223e7fd82dc6fb4ca2aea30f7f22`.

En ese mismo segundo, el stdout de service-a (2026-10-09T20:34:21.102Z) mostró esta línea:

```
{"level":"INFO",
 "message":"Pedido compuesto orderId=ord-001 sku=LAPTOP-X1 sufficient=true",
 "span_id":"6f4e7346db5b3b78",
 "trace_id":"77fa223e7fd82dc6fb4ca2aea30f7f22",
 "service":"service-a"}
```

Y el de service-b (20:34:21.100Z), esta otra:

```
{"level":"INFO",
 "message":"Inventario consultado sku=LAPTOP-X1 available=15",
 "span_id":"e971b873e68f1ccf",
 "trace_id":"77fa223e7fd82dc6fb4ca2aea30f7f22",
 "service":"service-b"}
```

El `trace_id` es el mismo en los dos logs. Los `span_id` son distintos, como debe ser, porque cada log se emitió dentro de su propio span de negocio.

En Loki encontramos dos líneas cuyo cuerpo empieza por `trace_id=77fa223e7fd82dc6fb4ca2aea30f7f22`, una por servicio, y la consulta por structured metadata `{service_name=~"service-a|service-b"} | trace_id="77fa223e7fd82dc6fb4ca2aea30f7f22"` devolvió esas mismas dos. Vale la pena anotar que la primera consulta, hecha a los pocos segundos de la petición, no trajo nada: el log ya estaba en stdout, pero todavía no había llegado a Loki.

Tempo, en la segunda lectura del mismo id, devolvió 13 spans. La tabla 2 muestra los principales.

Servicio	Spans
service-a	`GET /api/orders/{orderId}`, `order.compose`, `SELECT lab22.orders`, `GET` (cliente)
service-b	`GET /api/inventory/{sku}`, `inventory.lookup`, `SELECT lab22.inventory`

Tabla 2. Spans de la traza 77fa223e… agrupados por servicio (corrida local del 9 de octubre de 2026).

Además aparecen `Ping`, `Prepare statement` y `Execute prepared statement`. Esos los genera el agente en la capa JDBC; no son spans manuales duplicados. En la primera lectura por id solo venía service-a, y unos segundos después ya estaban los dos servicios. Es una carrera del ingester de Tempo: una búsqueda o una relectura encuentran la traza completa, y por eso `tests/verify-local.ps1` reintenta.

En Prometheus, con una ventana de 30 minutos en esa misma sesión, el SLI de disponibilidad de las rutas `/api/` dio 100, la tasa de errores 5xx dio 0 y `up{job="otel-collector"}` estaba en 1. Los archivos de evidencia quedaron en `docs/evidencias/`.

7. Dashboard

El dashboard aprovisionado se llama «Lab 2.2 — Pedidos e inventario» (uid `lab22-observability`). La figura 2 es la captura `docs/evidencias/grafana-dashboard.png`, tomada sobre los últimos 30 minutos de la sesión, después de la carga con k6.

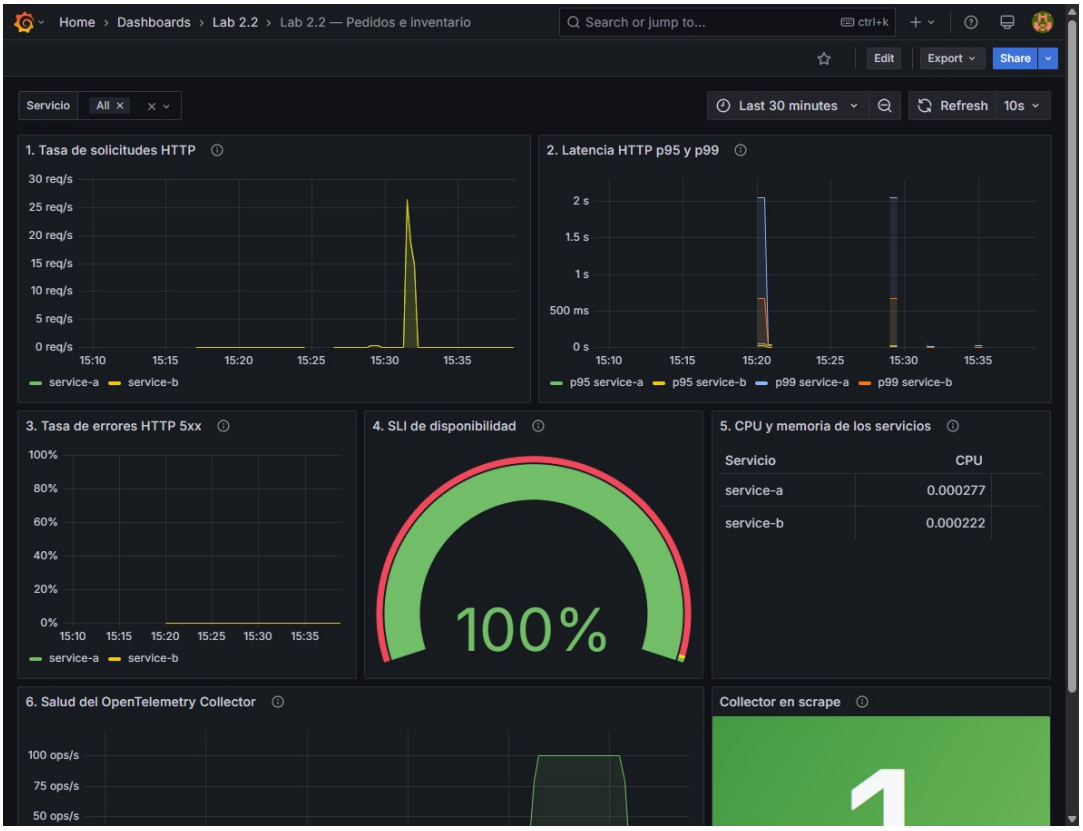


Figura 2. Dashboard local después del benchmark: el SLI marca 100 % y la tasa de errores 5xx está en cero (captura propia, 9 de octubre de 2026).

La tabla 3 resume lo que se vio en cada panel.

Panel	Qué observamos
1. Tasa HTTP	Pico cercano a 26 req/s en service-a y service-b hacia las 15:30 (hora local), que coincide con la ventana del benchmark.
2. Latencia p95 y p99	Picos de hasta unos 2 s. Esto no es el p99 de k6: `histogram_quantile` sobre `rate()` se dispara cuando el intervalo tiene pocas muestras y un máximo alto. El percentil válido de la prueba está en la sección 8.
3. Errores 5xx	Línea en 0 % para los dos servicios. Antes, como no existía la serie 5xx, Grafana mostraba "No data"; ahora la consulta agrega un cero derivado del total.
4. SLI	Gauge en 100 %.
5. CPU y memoria	En reposo, después de la carga: service-a con CPU reciente de 0,000277 y heap de 73 064 368 bytes; service-b con 0,000222 y 81 810 160 bytes. La métrica es `jvm_cpu_recent_utilization_ratio`, no el porcentaje de `docker stats` durante k6.
6. Collector	El job `otel-collector` estaba en 1. El panel de spans exportados y fallos comparte esa captura; los fallos se pintan en 0 cuando la serie de error todavía no existe.

Tabla 3. Lectura de los seis paneles del dashboard (elaboración propia, 2026).

La navegación entre señales la configuramos y la comprobamos con la API de Grafana después de reiniciar:

- **Loki:** derived field `TraceID` con la regex `trace_id=([0-9a-f]{32})`, destino Tempo y URL `${__value.raw}`.
- **Tempo:** `tracesToLogsV2` con la consulta `{service_name=~"service-a|service-b"} | trace_id=` más el id de la traza.
- **Prometheus:** exemplars hacia Tempo.

Aquí tropezamos con un error que vale la pena contar. El archivo de aprovisionamiento tiene que escribir `$$` delante de esas variables. Con un solo `$`, Grafana las borra al leer el YAML porque cree que son variables de entorno, y en algún momento la API llegó a devolver la consulta de logs con el id vacío. Después del escape, la API vuelve a devolver `${__trace.traceId}` y `${__value.raw}`. El enlace desde el `trace_id` del log y la consulta LogQL sí los probamos; el clic sobre un exemplar en la interfaz quedó sin ejercitar.

8. Benchmark

El script es `load-tests/run-benchmark.ps1`. Corrimos k6 0.57.0 en Docker con 5 usuarios virtuales durante 30 segundos y una pausa de 100 ms, rotando los pedidos `ord-001` a `ord-005` contra `http://host.docker.internal:8080`. La imagen, la base de datos y el Compose fueron los mismos en las dos pasadas; en la aplicación lo único que cambió fue `OTEL_ENABLED`.

En el escenario base el Collector siguió encendido, pero el proceso Java no cargaba el agente. El README del ejemplo propone apagar solo el Collector, y eso deja al SDK creando spans de todas formas, con lo que no se estaría midiendo el costo real de instrumentar.

En cada pasada, `docker stats` muestreó service-a, service-b y el Collector durante 40 segundos. El script `load-tests/analyze_overhead.py` aplica estas dos fórmulas:

overhead absoluto = instrumentado − baseline **overhead % = (instrumentado − baseline) / baseline × 100**

Métrica	Baseline	Instrumentado	Absoluto	%
Latencia p50 (ms)	8,6306	8,6653	0,0347	0,40 %
Latencia p95 (ms)	14,7352	16,4087	1,6735	11,36 %
Latencia p99 (ms)	21,6424	23,8405	2,1981	10,16 %
Throughput (req/s)	44,2853	43,2927	−0,9926	−2,24 %
Solicitudes	1331	1302	−29	−2,18 %
Tasa de errores	0	0	0	no aplica

Tabla 4. Latencia, throughput y errores medidos por k6 en las dos pasadas (corrida del 9 de octubre de 2026).

Contenedor	CPU base	CPU OTel	CPU %	Mem base	Mem OTel	Mem %
service-a	23,98 %	34,28 %	+42,97 %	293,04 MiB	403,50 MiB	+37,69 %
service-b	18,53 %	33,49 %	+80,69 %	266,23 MiB	431,27 MiB	+61,99 %
collector	1,58 %	0,91 %	−42,41 %	57,25 MiB	70,61 MiB	+23,33 %

Tabla 5. CPU y memoria por contenedor según docker stats (ventana de 40 s por pasada).

Todos los checks de k6 pasaron: 5320/5320 en el baseline y 5204/5204 en el instrumentado. En el baseline el check de `traceId` no revisa el cuerpo; en el instrumentado exige 32 caracteres hexadecimales distintos de cero, y también pasó.

9. Lectura del overhead

El p50 sube 0,03 ms, y en una sola pasada de 30 segundos eso es ruido. Donde sí se nota es en la cola: el p95 sube 1,7 ms y el p99 2,2 ms, alrededor del 11 % y del 10 %. El throughput baja un 2 % y no hubo errores HTTP.

La memoria de las JVM sube 110 MiB en service-a y 165 MiB en service-b, que es donde viven el agente, sus hilos y las colas de exportación. La CPU media de la ventana sube entre 10 y 15 puntos porcentuales, y service-b crece más porque atiende cada consulta de inventario con su JDBC. Estos números hay que leerlos con cuidado: la ventana de `docker stats` dura 40 segundos e incluye el arranque del contenedor recién recreado, así que no equivale a un perfil de estado estacionario de varios minutos.

La CPU del Collector bajó en la pasada instrumentada, pero no lo tomamos como un ahorro, porque el valor de partida es bajo y la muestra es corta. Su memoria sí sube, de 57 a 71 MiB, lo cual tiene sentido ahora que recibe lotes.

Si se quisiera bajar el costo sin perder visibilidad, las palancas que identificamos son estas:

- subir el intervalo de métricas por encima de 15 s;
- usar `parentbased_traceidratio` con un valor menor que 1 fuera del laboratorio;
- pasar la verbosidad del exporter debug del Collector a `basic`;
- seguir sin abrir spans manuales alrededor del SQL.

El batch del agente y el del Collector ya agrupan los envíos. Esta corrida no se repitió; los JSON de origen quedan en `load-tests/results/` por si se quiere recalcular.

10. Limitaciones y trabajo pendiente

Quedó hecho y comprobado en local: la compilación y `mvn test`, el Compose, un pedido real que pasa por service-b y MySQL, la traza con los dos servicios, los logs JSON con el mismo `trace_id`, las series en Prometheus, el dashboard con datos de la carga y el benchmark de la sección 8.

Queda pendiente, y no lo simulamos:

- **`terraform apply` en GCP y AWS.** No teníamos credenciales y en la máquina tampoco estaba el CLI de Terraform, así que `terraform validate` no se ejecutó. Los Collector de GCP y AWS sí pasaron `otelcol validate`.
- **El clic visual de un exemplar** de Prometheus hacia Tempo.
- **Repetir el benchmark** con más usuarios y varias pasadas. Los porcentajes de este informe no deben citarse como capacidad de producción.
- **El p99 del panel de Grafana** tampoco debe citarse como resultado de k6; el resultado de la prueba de carga es el de la tabla 4.

Versiones usadas: Spring Boot 3.5.16, MySQL 8.4, Prometheus 3.4.1, Grafana 11.6.0, Tempo 2.8.1, Loki 3.4.2 y k6 0.57.0. El detalle de cada parte está en la documentación del repositorio: `docs/analisis-repositorio-docente.md`, `docs/arquitectura.md`, `docs/instrumentacion-opentelemetry.md`, `docs/collector.md`, `docs/correlacion-cross-signal.md` y `docs/benchmark-overhead.md`.

Referencias

Grafana Labs. (s.f.). Configure the Loki data source: Derived fields. Grafana documentation. https://grafana.com/docs/grafana/latest/datasources/loki/configure-loki-data-source/

Grafana Labs. (s.f.). Configure the Tempo data source: Trace to logs. Grafana documentation. https://grafana.com/docs/grafana/latest/datasources/tempo/configure-tempo-data-source/

Grafana Labs. (s.f.). Provision Grafana. Grafana documentation. https://grafana.com/docs/grafana/latest/administration/provisioning/

Grafana Labs. (s.f.). Grafana k6 documentation. https://grafana.com/docs/k6/latest/

OpenTelemetry Authors. (s.f.). Collector. OpenTelemetry documentation. https://opentelemetry.io/docs/collector/

OpenTelemetry Authors. (s.f.). Java agent. OpenTelemetry documentation. https://opentelemetry.io/docs/zero-code/java/agent/

OpenTelemetry Authors. (s.f.). OpenTelemetry specification. https://opentelemetry.io/docs/specs/otel/

World Wide Web Consortium. (2021). Trace Context (W3C Recommendation). https://www.w3.org/TR/trace-context/